SPOT CHECK

SKATESPOTS FIELD MANUAL · №02

DATA — SHAPE, SERIALIZATION & THE MODEL

Everything we reasoned through while designing the `Spot` type and building `mockSpots`: what a data shape is, why it's a *contract*, what actually survives a trip through JSON, and how one skate spot lives at three different layers — the world, the wire, and the database.

11 chapters · 3 parts | Live JSON lab | Self-test included | Phase 1 — Designing to a data shape

SERIALIZATION

What actually travels between server and browser

01 What "serialization" actually means

CONCEPT

— THE IDEA

Your `Spot` lives in memory as a rich JavaScript object — it can hold a `Date`, nested objects, whatever. But a network can't send an "object." It can only send **bytes**, which for a web API means **text**. **Serialization** is turning your in-memory object into that flat text; **deserialization** (or parsing) is rebuilding an object from it on the other end.

In JavaScript this pair is `JSON.stringify()` (object → text) and `JSON.parse()` (text → object). Your API does the first; TanStack Query does the second for you in Phase 7.

> **The one law of UI-first:** your mock `Spot` must look exactly like the object that comes out of `JSON.parse()` on the client — *not* the object that existed on the server before serialization. Those two are not the same, and Chapters 03–04 are the places they differ.

— HOW TO STUDY

Open DevTools console and run `JSON.parse(JSON.stringify(obj))` on any object you build. Whatever comes back is the "wire shape" — the only shape your UI ever actually sees.

02 JSON's tiny type vocabulary

CONCEPT GOTCHA

— WHY

JSON only knows **six** value types. Everything you put in a `Spot` has to reduce to one of them, or it gets transformed / dropped on the way out:

JSON TYPE	YOUR SPOT FIELD	NOTE
`string`	`name`, `city`, `created_at`	Text in double quotes only
`number`	`id`, `rating`, `lat`, `lng`	IEEE-754 double — see Ch 05
`boolean`	`is_skateable`	`true` / `false`
`null`	an absent `photo`	The only "empty" — no `undefined`
`array`	`features`	Ordered list
`object`	`lat_lng`	Nested key/value

— THE GOTCHAS — WHAT IS NOT ON THAT LIST

- **No `Date`** — becomes a string (Ch 03).
- **No `undefined`** — a key whose value is `undefined` is *silently omitted* from the output object entirely. This is why "optional" and "null" are different things (Ch 08).
- **No enums, no classes, no functions, no `Map` / `Set`** — they flatten to plain objects or vanish.
- **No `BigInt`** — `JSON.stringify` throws on it outright.

> **Remember:** "JSON is text with six shapes." Anything richer than those six is your responsibility to encode on the way out and rebuild on the way in.

— HOW TO STUDY

In the console: `JSON.stringify({ a: undefined, b: null, c: new Date() })`. Watch `a` disappear, `b` stay, and `c` turn into a string. That single line teaches this whole chapter.

↗ MDN — JSON (the whole object)

↗ MDN — JSON.stringify (what it drops & transforms)

03 Dates don't survive the trip

BUG-IN-WAITING · CONCEPT

— WHAT CAME UP

We typed `created_at` as `string`, not `Date`. That looked odd at first — it *is* a date. But it's correct, and here's the machinery.

— WHY

A `Date` object has a hidden `.toJSON()` method. When `JSON.stringify` hits it, it calls that method, which returns an **ISO-8601 string** like `"2026-08-02T14:30:00.000Z"`. So by the time your spot reaches the browser, `created_at` is — and only ever was — a **string**. There is no `Date` on the client unless you rebuild one.

▸ LIVE: WATCH A DATE BECOME A STRING, THEN COME BACK

BEFORE — IN MEMORY

```
{
  id: 1,
  created_at: new Date(),
  photo: undefined
}
```

AFTER — JSON.STRINGIFY()

```
press Run →
```

> **The honesty rule for mock data:** because the real API sends a full ISO timestamp, your mock's `created_at` should be a full ISO string too — `"2026-08-02T14:30:00.000Z"`, not the date-only `"2026-08-02"`. A mock that's shaped differently from the API is a mock that lies, and the lie surfaces the day you swap in Phase 7.

— SOLUTION — THE PATTERN

Keep the transport type a `string`. Convert to a `Date` only at the moment you need date *math* or formatting, right where you use it:

```
// created_at stays a string everywhere it travels
const created = new Date(spot.created_at);   // re-hydrate on demand
created.toLocaleDateString();                // "8/2/2026"
```

— HOW TO STUDY

Run `new Date().toJSON()` in the console. Then `JSON.parse(JSON.stringify({ d: new Date() }))` and check `typeof result.d` — it's `"string"`. Prove the loss to yourself once and you'll never mistype a date field again.

↗ MDN — Date.prototype.toJSON

↗ ISO 8601 — the timestamp format

04 Location on the wire — and the lat/lng order trap

CONCEPT GOTCHA

— WHAT CAME UP

We chose `lat_lng: { lat: number; lng: number }` instead of two loose columns or a raw array. Behind that small choice is one of the most bug-prone corners of the whole project.

— WHY A NAMED OBJECT BEATS AN ARRAY

The database will store location as a PostGIS *point*, which an API can't send directly — it serializes to plain numbers. The question is only *how*. Three options:

SHAPE	LOOKS LIKE	RISK
Two fields	`lat: 42.3, lng: -71.0`	Location can get half-updated
Array	`[-71.0, 42.3]`	**Which one is which?**
Named object ✓	`{ lat: 42.3, lng: -71.0 }`	Self-documenting — chosen

— THE TRAP THE NAMED OBJECT SAVES YOU FROM

Different tools disagree about coordinate order, and they do it silently — both values are just numbers, so nothing errors; your pin just lands in the wrong ocean.

TOOL	ORDER IT EXPECTS
Leaflet (Phase 3)	`[lat, lng]` — latitude first
GeoJSON / PostGIS (Phase 6, 10)	`[lng, lat]` — longitude first

> **Boston is `lat 42.36, lng -71.06`.** Swap them and you get `lat -71, lng 42` — off the coast of Western Sahara. That's the exact class of bug the earlier `{ lat: 4, lng: 4 }` placeholder would have hidden: numbers that parse fine and point nowhere real. Naming the fields `lat` and `lng` means the order can never be ambiguous at the callsite.

— HOW TO STUDY

Look up your four spots' real coordinates and confirm each lands in Boston on a map. Then read one line of the Leaflet quick-start and one line of the GeoJSON spec and note that they order coordinates *oppositely* — that contradiction is the thing to remember.

- ↗ GeoJSON RFC 7946 — position is [lng, lat]
- ↗ Leaflet — LatLng is (lat, lng)
- ↗ PostGIS — ST_MakePoint(x=lng, y=lat)

II MODELING THE SPOT

The specific decisions baked into your type

05 id: number or string?

CONCEPT

— WHAT WE DECIDED

You chose `id: number`. That's a real fork in the road, and it ripples into your React `key` props, your `/api/spots/[id]` route, and your Prisma primary key. Here's the trade so you can defend it.

	`NUMBER` (AUTO-INCREMENT)	`STRING` (CUID / UUID)
DB default	`serial` / `autoincrement()`	`cuid()` / `uuid()`
Readability	Great — `1, 2, 3`	Opaque — `clx1a…`
Guessable?	Yes — `/spots/2` invites enumeration	No — practically unguessable
Made where?	By the database on insert	Can be made on the client too

For a learning project, `number` is a perfectly good call — it keeps Prisma Studio readable and the URLs friendly. Just know the trade-off you accepted: sequential ids leak "how many spots exist" and are trivially enumerable. If this ever went public with private data, that's when you'd revisit it.

> **Contract consequence:** whatever you pick, your mock ids must be that type. `id: 1` and `id: "1"` are different contracts, and `===` comparisons in your click handlers will care.

— HOW TO STUDY

Skim the Prisma `@id` docs and note the two default generators. Ask yourself: "who assigns this value, and when?" That question — not readability — is the real axis.

↗ Prisma — defining an id field

06 Union types as enums

CONCEPT

— WHAT YOU BUILT

Instead of typing `spot_type: string`, you named the exact allowed values:

```
export type SpotType = "park" | "street" | "diy";
export type Status   = "active" | "pending" | "closed";
export type Feature  = "ledge" | "rail" | "plaza" | "stairs" | "skatepark";
```

— WHY THIS IS THE BETTER MOVE

A **string-literal union** turns a whole category of runtime bugs into compile-time errors. `spot_type: "streat"` (typo) is a red squiggle immediately, not a filter that silently matches nothing in Phase 5. Your editor also autocompletes the legal values, so you never have to remember them.

> **This is a contract, not a convenience.** When Phase 5's filter compares `spot.spot_type === activeFilter`, it's comparing against these *exact strings*. And in Phase 6 your Postgres/Prisma enum must list these same values, character for character. Change one here and three phases downstream have to agree.

— THE WATCH-OUT

A union only guards code the compiler can see. Data arriving from the *network* in Phase 7 is typed `any` until you validate it — a row with `spot_type: "vert"` in the DB would sail past TypeScript. That's exactly why Phase 8 introduces **Zod**: to re-check these unions at the runtime boundary where the compiler can't reach.

— HOW TO STUDY

In a scratch `.ts` file, assign `const t: SpotType = "vert"` and read the error — it lists your legal values back to you. Then widen it to `string` and watch the safety disappear.

↗ TypeScript — literal & union types

07 spot_type vs features — category vs attributes

CONCEPT

— THE DECISION WORTH BEING PROUD OF

The roadmap floated one flat list — `park | street | ledge | rail | stairs | bowl | DIY`. You split it into **two** dimensions instead:

SPOT_TYPE — WHAT KIND OF PLACE

`"park" | "street" | "diy"`

exactly one per spot

+

FEATURES — WHAT'S THERE

`("ledge" | "rail" | "plaza" | …)[]`

many per spot

— WHY IT'S THE STRONGER MODEL

Those original values weren't actually the same *kind* of thing. "Street" is a category; "ledge" and "rail" are things a street spot *has* — and it can have several at once. Cramming them into one field forces a false choice ("is Eggs a street spot or a ledge spot?" — it's a street spot *with* ledges and stairs, which your model says cleanly: `spot_type: "street", features: ["ledge","stairs"]`).

This is the classic modeling question: **"is this a single category, or a set of attributes?"** Single category → one enum field. Set of independent traits → an array (or, later, a related table). Getting this right now means the Phase 5 filters can be richer: filter by *type*, by *feature*, or both.

> **The seam for later:** in Phase 6 this two-axis design maps naturally to the database — `spot_type` becomes an enum column, and `features` becomes either a Postgres array column or a full one-to-many `Feature` table (a Phase 9 / stretch decision). You built the flexibility in for free.

— HOW TO STUDY

Take three real spots you know and write each as `{ spot_type, features }`. If you ever feel the urge to put the same value in both, that's the signal your two axes have blurred — re-separate them.

↗ React — Thinking in React (the same "one thing vs many" instinct, for UI)

08 Optional fields = the nullability contract

CONCEPT GOTCHA

— WHAT YOU DECIDED

You marked `description?` and `photo?` optional, and left the rest required. Each `?` is a promise to the rest of the app: *"this field might not be here — you must handle its absence."*

— THE THREE WAYS "NO PHOTO" CAN LOOK

This is subtle and worth pinning down, because `?` in TypeScript and the JSON on the wire don't line up perfectly:

ON THE WIRE	IN TS AFTER PARSE	MEANING
key absent	`photo === undefined`	"never set" — what `?` models
`"photo": null`	`photo === null`	"explicitly empty" — DB NULL
`"photo": ""`	`photo === ""`	"set, to nothing" — a real string!

> **Your mock uses `photo: ""`.** That's a *present, empty* string, which is a different case from the optional-and-absent that `photo?` describes. Pick one convention — "missing photo means the key is absent" *or* "means `null`" — and make the mock and the API agree, so the Phase 4 details panel only has to check one thing before rendering the image.

— WHY IT MATTERS DOWNSTREAM

Every optional field is a branch your components must write: `{spot.photo ? <img …/> : <Placeholder/>}`. Deciding nullability now, at the contract, is deciding how many of those branches exist. It also lines up with the database: an optional field is a **nullable column**; a required one is `NOT NULL`. The `?` you type today is a schema constraint you're pre-committing to.

— HOW TO STUDY

Console: compare `obj.photo === undefined`, `=== null`, and `=== ""` for each of the three shapes above. Then read how `JSON.stringify` treats a key whose value is `undefined` (it drops it) vs `null` (it keeps it) — that's why the first two rows differ.

- ↗ TypeScript — optional properties
- ↗ MDN — falsy values (why `""`, `null`, `undefined` all read as false)

09 Naming consistency & type/data lockstep

BUG AVOIDED · CONCEPT

— WHAT HAPPENED

The shape briefly mixed conventions — `spot_type`, `lat_lng`, `is_skateable` in `snake_case` but `createdAt` in `camelCase`. You unified on `snake_case` (`created_at`) — and, crucially, you changed it in **both** the type *and* the data at once.

— WHY THE LOCKSTEP MATTERS

The type file and the mock file are two halves of **one contract**. If you rename `createdAt` → `created_at` in the data but not the type, TypeScript screams: the object is missing a required `createdAt` and carries an unknown `created_at`. You moved them together, so the compiler stayed green — that's the discipline: *a shape change is never done until the type and every producer of that shape agree.*

> **Why we even checked the compiler:** a clean `tsc` is the cheap proof that type and data still describe the same thing. When you renamed the field, "zero diagnostics" was the evidence the contract was still whole. Treat a green compile as a passing test of your data shape.

— THE ONE HONEST CAVEAT ABOUT SNAKE_CASE

JavaScript's own convention is `camelCase`; `snake_case` keys are common in APIs (and match SQL column names). Either is fine — **consistency is the whole point**. Just know that if your Postgres columns end up `snake_case` and some JS wants `camelCase`, the translation happens in exactly one place: the serialization boundary (Ch 11). Choosing `snake_case` end-to-end means *no* translation — one fewer moving part.

— HOW TO STUDY

Deliberately rename one field in the mock but not the type, save, and read the two errors TypeScript gives (missing key + excess key). Then fix the type. That error pair *is* the lockstep lesson — feel it once on purpose.

↗ TypeScript — excess property checks

III

ENTITY → MODEL

One spot, living at three layers of the stack

10 One spot, three representations

CONCEPT

— THE BIG IDEA

"A skate spot" is not one thing in your codebase — it's the *same concept* wearing three different costumes depending on where it stands. Confusing the layers is the root of most data bugs; separating them is most of software architecture.

LAYER	NAME FOR IT	WHAT IT IS	LOCATION = ?
1. Entity	Domain entity	The real ledge by the water. A concept, not code.	a physical place
2. Wire	DTO / your `Spot` type	The JSON your API sends & your UI renders.	`{ lat, lng }`
3. Store	Data model / schema	The Prisma model & Postgres table it lives in.	`geography(Point)`

▸ THE SAME SPOT, THREE COSTUMES

```
1 · ENTITY  "Copley Library ledges" — a real spot in Boston
    │
    ▼  you decide what's worth capturing
2 · DTO (types/spot.ts)     the contract — Chapters 05–09
    { id: 3, name: "Copley Library",
      spot_type: "street", features: ["plaza"],
      lat_lng: { lat: 42.35, lng: -71.08 },
      created_at: "2026-08-02T…Z" }
    │
    ▼  mostly a rename + type-swap
3 · DATA MODEL (schema.prisma → Postgres)
    model Spot {
      id          Int      @id @default(autoincrement())
      name        String
      spot_type   SpotType            // Postgres enum
      location    Unsupported("geography(Point,4326)")
      created_at  DateTime @default(now())  // timestamptz
    }
```

> **Why UI-first works because of this:** you're building layer 2 (the DTO) first, by hand, against mock data. Layer 3 (the model) comes in Phase 6 and is shaped to *match* what layer 2 already promised. That's the whole bet from the roadmap — get the middle layer right and the bottom layer just conforms to it.

— HOW TO STUDY

Take one spot and write it out at all three layers on paper. Circle the fields that *change representation* between layer 2 and layer 3 (location, created_at, the enums). Those circles are Chapter 11.

↗ Fowler — Data Transfer Object (the "wire" layer, named)

11 The mapping — and where the transformation lives

CONCEPT GOTCHA

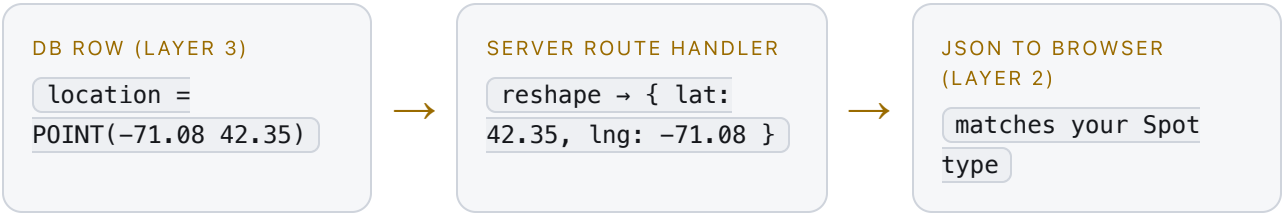
— THE FIELD-BY-FIELD MAP

Every field either passes through unchanged or gets **transformed** as it crosses from the database (layer 3) to the wire (layer 2). Here's your `Spot`, mapped:

DTO FIELD (CH 05–09)	POSTGRES / PRISMA	TRANSFORM?
`id: number`	`Int @id @default(autoincrement())`	pass-through
`name: string`	`String` / `text`	pass-through
`spot_type: SpotType`	Postgres `enum`	same values, different world
`features: Feature[]`	`text[]` or related table	array ↔ column/rows
`lat_lng: {lat,lng}`	`geography(Point,4326)`	**real transform** — Ch 04
`created_at: string`	`DateTime` / `timestamptz`	**ISO string ↔ timestamp** — Ch 03
`description?: string`	nullable `text`	optional ↔ NULL — Ch 08

— WHERE THE TRANSFORM PHYSICALLY HAPPENS

Not in the browser, and not in the database — in **your route handler on the server** (Phase 7's `GET /api/spots`). That's the one place both worlds are visible at once:



This gap between "how it's stored" and "how it's used" has a name: **impedance mismatch**. Your job at the server boundary is to absorb it, so the UI never sees a `geography` blob and

the database never sees a JS `Date`.

> **The PostGIS reality check:** Prisma can't fully model a `geography` column, so Phase 6/10 you'll declare it as `Unsupported(...)` and read it with a raw, *parameterized* query (`ST_X` / `ST_Y` to pull `lng` / `lat` back out). That's not Prisma failing — it's the exact seam where layer 3 refuses to auto-map to layer 2, and you step in. Your `{ lat, lng }` DTO is the target that raw query fills.

— HOW TO STUDY

For each row in the map above, answer "pass-through or transform?" without looking. The three transforms — location, date, optional/NULL — are precisely the three things Parts I & II drilled. That's not a coincidence; this manual was built backwards from this table.

↗ Prisma — raw & parameterized queries

↗ PostGIS — geography basics

↗ Object–relational impedance mismatch

A Working principles from this session

1. **The mock is the wire shape.** Design mock data to match what `JSON.parse` yields on the client, not the server object. *(Ch 01, 03)*
2. **JSON is text with six shapes.** Anything richer — dates, geometry, enums — is transformed or dropped. Know which. *(Ch 02)*
3. **Name coordinates, never order them.** `{ lat, lng }` beats `[a, b]` because tools disagree on order silently. *(Ch 04)*
4. **Model categories and attributes separately.** One-of → enum; many-of → array/relation. *(Ch 07)*
5. **Optional is a promise, not a default.** Every `?` is a branch downstream and a nullable column below. *(Ch 08)*
6. **Type and data move in lockstep.** A shape change isn't done till the compiler is green again. *(Ch 09)*
7. **One spot, three layers.** Entity → DTO → data model; the server boundary absorbs the mismatch. *(Ch 10, 11)*

B Self-test

Say the answer aloud *before* opening each — the explaining is what makes it stick.

Q1 Why is `created_at` typed `string` and not `Date` ? ▸

Q2 What happens to a key whose value is `undefined` when you `JSON.stringify` ? ▸

Q3 Boston is around `lat 42.36, lng -71.06` . Why is a named `{lat,lng}` object safer than `[42.36, -71.06]` ? ▸

Q4 Why split `spot_type` from `features` instead of one flat enum? ▸

Q5 Name the three ways "no photo" can appear, and which one `photo?` describes. ▸

Q6 You rename a field in the mock but not the type. What does TypeScript say, and what's the lesson? ▸

Q7 Name the three layers a single spot lives at. ▸

Q8 Which three `Spot` fields genuinely *transform* between database and wire, and where does that happen? ▸

C Study links — go deeper

— SERIALIZATION & JSON

- ↗ MDN — JSON.stringify (what it transforms & drops)
- ↗ MDN — Date.toJSON
- ↗ ISO 8601 timestamps

— TYPING THE SHAPE

- ↗ TypeScript — Everyday Types (unions, optionals)
- ↗ Zod — runtime validation for the network boundary (Phase 8)

— THE DATA MODEL & GEO

- ↗ Prisma — data model & ids
- ↗ PostGIS — geography basics
- ↗ GeoJSON RFC 7946 (coordinate order)

— ARCHITECTURE CONCEPTS

- ↗ Fowler — Data Transfer Object
- ↗ Object–relational impedance mismatch